

ATANDA COMMAND CENTRE — CURRENT-STATE PDD

4-Part ATLAS Project Definition Document · v3.0 · May 2026

****Document ID:**** ATANDA-CC-PDD-CURRENT-2026-003

****Supersedes:**** JNGL-AC-ATANDA-CC-2026-001 (MVP ATLAS PDD · May 2026)

****Scope:**** Captures the codebase as deployed, the full feature inventory, the user benefits each feature delivers, and a side-by-side comparison with the last published PDD.

****Status:**** PUBLISHED · CURRENT-STATE · COMPREHENSIVE

PART 1 — EXECUTIVE SUMMARY

1.1 What the product is, in one paragraph

ATANDA Command Centre is a subscription portal that wraps the **FORGE.BONSAI HARNESS** — a PDD-blueprint application whose engines are an ordered sequence of atomic prompts (F1 through F7, plus F6-VDJ, F8 Code DJ, ATLAS J, and PFP) that drive a Large Language Model through a deterministic *prompt !' SPC !' PDD !' certified MVP-PDD pipeline*. A creator signs in, opens a session, and walks the HARNESS from a raw prompt to a SPARTAN-certified MVP PDD with a public verification URL. The HARNESS itself is the instruction layer; the SPCs and PDDs are what it produces on behalf of the user.

1.2 Business model

- **Three subscription tiers** (Explorer free !' Practitioner !' Architect !' Institution) sold via Stripe Checkout + Stripe Billing portal, monthly or yearly.
- **Two per-project standalone SKUs** that sit outside the subscription: - **Ingestion Credit** (one credit = one document !' PWDD session, regardless of tier). **Advanced Cartridge** (\$499.99 / project) for users arriving with multi-document context, prior SPCs, and live codebase / database links.

1.3 Headline change versus the last PDD

The previous PDD targeted a Next.js + Supabase + Sanity CMS + Cloudflare R2 + Vercel build, executed in Cursor by a single developer in four weeks, with 7 HARNESS features (F1–F7). The current build is a pnpm monorepo on Express + Drizzle ORM + PostgreSQL + Clerk + Stripe, executed against the Replit artifact system, with **11 HARNESS engines** (F1–F7, F6-VDJ, F8 Code DJ, ATLAS J, PFP), **two per-project billing SKUs**, **drift-gated codebase generation**, and a **cross-provider LLM test rig** that the original PDD did not contemplate. The platform thesis is intact; the implementation has been entirely re-platformed and expanded.

PART 2 — CURRENT STACK & CODEBASE TOPOGRAPHY

2.1 Runtime stack

Layer	Technology	Notes
Workspace	pnpm monorepo · Node.js 24 · TypeScript 5.9	One repo, multiple artifacts.
API	Express 5	Bundled via esbuild as CJS.
Auth	Clerk (Replit-managed)	JIT-syncs local users on first request.
DB	PostgreSQL + Drizzle ORM	No RLS — app-layer authorization on `req.localUser.id`.
Validation	Zod v4 + drizzle-zod	Schemas generated from OpenAPI.
API contract	OpenAPI 3.1 + Orval	Generated Zod schemas and React Query hooks.
Billing	Stripe (subscriptions + one-time)	Idempotent webhook with raw-body parser.
LLM	Claude Sonnet 4 via Replit AI Integrations proxy	Provider switching across 3 backends.
Frontend	React + Vite + Wouter + TanStack Query	Single command-centre artifact.
Observability	pino logs · Sentry (API + web) · per-engine telemetry table	DSN-gated, no-op in dev.
Email	Resend (with console dry-run fallback)	Transactional only.

2.2 Registered artifacts

ID	Kind	Title	Directory
api-server	api	API Server	artifacts/api-server
mockup-sandbox	design	Canvas	artifacts/mockup-sandbox
command-centre	web	ATANDA Command Centre	artifacts/command-centre

2.3 Where things live

- API contract: [lib/api-spec/openapi.yaml](#) (single source of truth)
- Generated Zod + React Query: [lib/api-zod](#), [lib/api-client-react](#)
- DB schema: [lib/db/src/schema/*.ts](#) (one table per file)
- Express app + middlewares: [artifacts/api-server/src/app.ts](#)
- Auth bridge: [artifacts/api-server/src/lib/auth.ts](#)
- Tier + rate-limit gates: [artifacts/api-server/src/lib/tier.ts](#)
- HARNESS engines: [artifacts/api-server/src/engines/{f1..f7,f6vdj,f8codedj,atlas-crystallise,pfp}.ts](#)
- Frontend workspaces: [artifacts/command-centre/src/components/workspaces/F{1..8}*.tsx](#)

PART 3 — FEATURE INVENTORY & USER BENEFITS

3.1 Core HARNESS pipeline (F1 !' F7)

#	Engine	What it does for the operator	User benefit
F1	Test Your Prompt	Diagnoses a raw prompt against the 7-pillar JCSE rubric.	Instantly tells the user <i>*why*</i> a prompt is weak before any LLM cost is incurred.
F2	Build Atomic Prompt	Walks the user through a 7-pillar wizard that emits an Atomic Prompt.	Turns a vague idea into a structurally complete, model-ready instruction.
F3	Build an MA	Cultivates a CELL Micro-Agent Birth Package via streamed responses.	Operationalises the prompt into a deployable agent definition.
F4	Convert to Micro PDD	Compresses the MA into a CELL Micro PDD.	Produces a compact, reusable design document for the agent.

F5	Build an SPC	Runs the ATLAS FORGE Q&A SPC builder.	Generates a full Single Page Concept that downstream teams can act on.
F6	Draft an ATLAS PDD	Produces the 4-Part ATLAS PDD (cheat sheet, exec summary, worksheet, implementation).	Delivers an investor- and engineer-grade product definition in one pass.
F7	Compress to MVP PDD	Runs the SPARTAN SCM 7-step compression to a SPARTAN-certified MVP PDD with a public verification URL.	Yields a build-ready, certifiable MVP plan the user can hand to any developer or AI IDE.

3.2 Side-step engines

Engine	Tier	Purpose	User benefit
F6-VDJ	Practitioner	Recommends the best VIBE / AI-IDE pairing for the drafted PDD.	Removes guesswork about which AI coding environment fits the project.
F8 Code DJ	Architect	Consumes a SPARTAN-certified MVP PDD and emits a CODEBASE_BUNDLE ("d12 files + manifest) for a chosen platform (Next.js · Vite · Express · Expo · pnpm monorepo).	Turns a certified PDD into a real scaffolded codebase, gated by drift checks.
ATLAS J	Practitioner	Crystallises an existing 4-Part ATLAS PDD into a typed JSON view with stable per-phase prompt IDs (P-RED-001 ...), CLASS A/B/C classification, dependency edges, stack, routes, deploy target.	Lets downstream tooling, dashboards, and automation consume the PDD as data, not prose.
PFP	Practitioner	Cross-references the MVP PDD against the generated CODEBASE_BUNDLE and emits a drift report with a fixed 7-code taxonomy (SPEC_DRIFT, PDD_ORPHAN, UNAUTHORIZED_EXTENSION, CIRCULAR_DEPENDENCY, SEMANTIC_DRIFT, OVER_SPECIFICATION, AMBIGUOUS_OUTPUT), 4-rung severity ladder, FCI score, and verdict.	Catches *spec-vs-code* drift before the operator ships a build that no longer matches its own PDD.

DE-SPC	Practitioner + ASPE badge	Digitally evolves an existing SPC into a refined SPC.	Lets advanced operators iterate SPCs without restarting the pipeline.
--------	---------------------------	---	---

3.3 Drift gate on F8 Code DJ

F8 refuses to scaffold a codebase if the latest PFP report for the same MVP PDD has any critical findings. The refusal is HTTP 409 with `code: "DRIFT_GATE"` and the PFP metadata embedded in the response body. The operator overrides only by re-submitting with `acknowledgeDrift: true`. Counts and verdict are **recomputed server-side from findings** before persistence so a self-contradictory model output cannot bypass the gate. The lookup filters by JSON path with `LIMIT 1`, so a busy session cannot push the relevant report past a row-window.

3.4 Ingestion path (per-project, tier-independent)

- `POST /api/billing/ingestion/checkout` opens a one-time Stripe Checkout for an Ingestion Credit.
- On `checkout.session.completed`, the webhook inserts an `ingestion_credits` row (`status='available'`).
- `POST /api/ingest` accepts a file (PDF / DOCX) or pasted text, extracts via `pdf-parse / mammoth`, normalises with Claude into `{detectedTitle, sourceDockKind, summary, seedPrompt}`, and **atomically claims one available credit** (`UPDATE ... FOR UPDATE SKIP LOCKED`).
- `POST /api/ingest/:id/start-session` opens a HARNESS session with `origin='ingested'`, F1 unlocked, others locked.
- The session pays once at ingest; subsequent F1 !' F7 runs are not re-charged.

3.5 Advanced Cartridge (per-project premium)

- `$499.99` SKU for projects arriving with multi-document context, prior SPCs, and/or live codebase / database links.
- Wizard at `/cartridge`: - **Step 1 (REQUIRED, server-validated)**: Define Project Scope — "e 80 chars scope, "e 12 chars outcome, non-empty project name. Server returns `code: "SCOPE_REQUIRED"` on violation. - **Step 2 (optional)**: Upload files, prior SPCs, descriptor-only Git / database links (no credentials).
- `POST /api/cartridge` claims one credit, summarises each doc with the user's preferred provider, persists `cartridge_packages` + children atomically, links the credit.
- `POST /api/cartridge/:id/start-session` unlocks F1 !' F7 as AVAILABLE in one shot.
- **Cartridge context is the protected instruction layer**. Loaded once per request in `engines/shared.ts#calllllm` (60-second LRU cache, max 24 KB) and prepended to every system prompt, scope first, fenced as `=== CARTRIDGE CONTEXT (authoritative · do not contradict) ===`.

3.6 Quest badges

Badge	Trigger	Storage
-------	---------	---------

ASPE	Live count of SPCs.	Computed, no row.
AISA	Live count of MAs.	Computed, no row.
AISE	URL-verified deployment, SSRF-hardened verifier.	Persisted in `command_centre_badges`.
AISA_PWDD (Advanced)	"e 3 PWDD-stage projects across manual + ingested + cartridge origins.	Computed live, no row.
AISE_BUILD (Advanced)	`POST /api/me/badges/engineer` with URL + evidence note ("d 500 chars). Same SSRF-hardened verifier.	Persisted with explicit `badgeId='AISE_BUILD'`.
Context Craft (7 pillars: S, R, I, D, F, E, C)	Auto-awarded fire-and-forget when an F1/F2 JCSE pillar sub-score "e 6.	Persisted in `context_craft_badges`, upserted with `GREATEST(bestScore, new)`.

3.7 Account lifecycle

`POST /api/me/delete` is **external-first, local-last**: cancel Stripe sub (hard-fail 502 if unreachable while a subscription id exists; benign on 404) `!clerkClient.users.deleteUser` (hard-fail 502 on error) `!' cascade-delete localUsers row !'` best-effort goodbye email. This ordering prevents orphaned Stripe subs and stale-token JIT re-creation.

PART 4 — ARCHITECTURE & OPERATIONAL DECISIONS

4.1 Contract-first API

Every endpoint is declared in `openapi.yaml` first. The server uses generated Zod for request bodies. The client uses generated React Query hooks. Response shapes are typed by generated TS interfaces; response-side Zod is not generated by Orval.

4.2 App-layer authorization, no RLS

Every Drizzle query that touches user-owned data filters on `req.localUser.id`. There is no Postgres RLS net. This is enforced as a code-review rule on every new route.

4.3 Middleware order

The Express app mounts middlewares in a strict order: Sentry init !' Clerk proxy path `/api/webhooks/stripe` (raw body) !' CORS !' JSON / urlencoded parsers `ClerkMiddleware` !' `/api` routes. Re-ordering breaks either Stripe signature verification or Clerk auth.

4.4 Stripe webhook idempotency

Every event id is recorded in `stripe_webhook_events` (PK on `event_id`) on receipt. Replays return `{ok: true, replay: true}` without re-executing handlers. If a handler throws, the idempotency row is rolled back so Stripe can legitimately retry. Unknown price IDs throw `UnknownPriceError` !' HTTP 400 with `{priceId, customerId}` logged.

4.5 Tier gating + escalation bypass

`requireTier("PRACTITIONER")` is enforced on F5 / F6 / F7. If the request body has a `sessionId` whose `harness_escalations` row exists, the tier gate is bypassed for that session.

4.6 Daily rate-limit counters

`f{1..8}_today` columns on the subscriber row. A guarded `/api/cron/reset-harness-limits` (header `x-cron-secret`) resets them nightly. Adding a new rate-limited engine without updating this reset handler permanently locks tier holders after their first day.

4.7 Per-engine telemetry

Every Claude call writes a row to `harness_engine_runs` via the optional `RunContext` argument to `callClaude` / `callClaudeJson`. Telemetry failures log a warning but never break the engine response.

4.8 LLM provider switching + cross-provider test rig

The `@workspace/api-server` provider-switching suite is **30 tests across F1 + 9 other engines × 3 providers**. It replays cached LLM responses from `test/__fixtures__/llm/<provider>/*.json` so the full suite runs in ~15 seconds with no network. The cache key normalises UUIDs and ISO timestamps so fresh per-run IDs don't bust the cache. A nightly `nightly-cross-provider.yml` GitHub Action runs `test:live` against all three providers, detects byte-level fixture drift with `git diff`, uploads the rewritten fixtures, and opens a `fixture-drift` issue.

4.9 AISE URL verifier — SSRF hardening

`lib/badges.ts#headOk` resolves the hostname and refuses RFC1918, loopback, link-local, metadata (169.254.169.254), CGNAT, and multicast addresses, and rejects anything that is not `https:`. The same verifier is reused by the `AISE_BUILD` senior badge endpoint.

PART 5 — COMPARATIVE ANALYSIS ·

CURRENT-STATE PDD vs PRIOR PDD

5.1 Stack delta

Dimension	Prior PDD (May 2026 SPARTAN MVP)	Current Build (this PDD)
Repo shape	Single Next.js app inside existing ATANDA repo	pnpm monorepo with separate api / web / sandbox artifacts
Framework	Next.js 14 (App Router)	Express 5 (API) + React + Vite + Wouter (web)
Database	Supabase Postgres with RLS + auth.users trigger	PostgreSQL + Drizzle ORM, app-layer auth on `req.localUser.id`, no RLS
Auth	Supabase Auth + Supabase JS client	Clerk (Replit-managed) with JIT local-user sync
Backend compute	Supabase Edge Functions (Deno)	Express handlers bundled by esbuild as CJS
CMS	Sanity CMS for marketing pages	None — landing + `/pricing` rendered in-app
Object storage	Cloudflare R2	Replit App Storage when needed
Hosting target	Vercel	Replit deployments
IDE assumption	Cursor (TARANTULA mandate)	Replit (Agent + Workspaces)
Timeline assumption	Single developer, 4 weeks	Continuous build; current state captures the post-cartridge, post-PFP, post-Code-DJ era
LLM client	Direct Anthropic SDK from Edge Function	`@workspace/integrations-anthropic-ai` via Replit AI Integrations proxy with provider switching
Testing	Manual + E2E launch checklist	Vitest provider-switching suite (30 tests, cached fixtures) + nightly cross-provider drift action
Observability	Implicit (Supabase logs)	pino + Sentry (API + web) + per-engine telemetry table

5.2 Feature delta

Capability	Prior PDD	Current Build	Net change
F1 Test Your Prompt	'	'	Same
F2 Build Atomic Prompt	'	'	Same
F3 Build an MA (SSE)	'	'	Same
F4 Convert to Micro PDD	'	'	Same
F5 Build an SPC	'	'	Same
F6 Draft an ATLAS PDD	'	'	Same
F6-VDJ recommendation	'	'	Same
F7 Compress to MVP PDD	'	'	Same
DE-SPC (digital evolution)	not in prior PDD	' (Practitioner + ASPE badge)	**ADDED**
F8 Code DJ (PDD !' codebase scaffold)	not in prior PDD	' (Architect tier, 5 platforms)	**ADDED**
ATLAS J (PDD !' typed JSON)	not in prior PDD	' (Practitioner)	**ADDED**
PFP (drift detection)	not in prior PDD	' (Practitioner, gates F8)	**ADDED**
Ingestion engine (doc !' PWDD session)	not in prior PDD	' (per-project credit, tier-independent)	**ADDED**
Advanced Cartridge (\$499.99 / project)	not in prior PDD	' (multi-doc + Git/DB links, scope-validated)	**ADDED**
Senior badges (AISA_PWDD, AISE_BUILD)	not in prior PDD	' (live + persisted, SSRF-hardened)	**ADDED**
Context Craft 7-pillar mini-quest badges	not in prior PDD	' (auto-awarded on JCSE breakdowns)	**ADDED**
Cross-provider LLM test rig	not in prior PDD	' (3 providers × cached fixtures · nightly drift)	**ADDED**
Per-engine telemetry table	not in prior PDD	' (`harness_engine_runs`, best-effort)	**ADDED**
Stripe webhook idempotency table	implicit	' (`stripe_webhook_events`, rollback-on-throw)	**HARDENED**
AISE URL verifier SSRF hardening	not specified	' (refuses RFC1918, loopback, metadata, etc.)	**ADDED**
Subscriber Tier Enforcement	' (middleware)	' (`requireTier` per route + escalation bypass)	Equivalent, more granular

5.3 Billing delta

Aspect	Prior PDD	Current Build
Pricing tiers	Explorer / Practitioner / Architect / Institution	Same four tiers
Per-tier price envs	Single subscription per tier	<code>`STRIPE_PRICE_{PRACTITIONER,ARCHITECT}__{MONTHLY,YEARLY}`</code>
Per-project SKUs	None	<code>`STRIPE_PRICE_INGESTION_PROJECT`</code> and <code>`STRIPE_PRICE_CARTRIDGE_PROJECT`</code>
Webhook hardening	Not specified	Raw-body parser mounted before JSON; idempotent on <code>`event_id`</code> ; unknown-price loud failure
Account deletion path	Not specified	External-first: Stripe !' Clerk !' local cascade !' goodbye email

5.4 Terminology delta — IPDD vs PWDD

The bare term "PDD" was overloaded in the prior PDD: it meant both the human-authored INPUT to ingestion **and** appeared as a suffix on the HARNESS-produced OUTPUTS (Micro PDD, ATLAS PDD, MVP PDD). The current build resolves this ambiguity:

- **IPDD = INPUT.** "Ingestion Product Design Document." A human-authored, pre-ingestion artefact (PDD, SDD, concept note, spec, brief). The Ingestion Engine consumes it.
- **PWDD = OUTPUT.** "PromptWare Design Document." The HARNESS-certified, post-ingestion outcome of a session seeded by ingesting an IPDD.
- **Qualified output PDDs are untouched.** Micro PDD (F4), ATLAS PDD (F6), MVP PDD (F7) keep their names — they are always *qualified* by a prefix and cannot collide with the input term.
- **Internal enum is untouched.** `sourceDocKind` value `product_design_document` is unchanged — no migration, no codegen drift.

5.5 What the prior PDD got right and the current build kept

- The **four-tier subscription model** (Explorer / Practitioner / Architect / Institution).
- The **7-feature core HARNESS pipeline** (F1 through F7) and the **F6-VDJ VIBE recommendation** as a side-step.
- The **escalation bridge** from F3 !' F5 for Practitioner-tier sessions that need the full SPC pipeline.
- The **tier-gated rate-limit counters** living on the subscriber row, reset by a cron.
- The **HARNESS-as-instruction-layer** framing: the engines are atomic prompts that drive a model deterministically; the SPCs and PDDs are what the model produces, not the engines themselves.

5.6 What the prior PDD assumed and the current build replaced

Cursor-as-IDE assumption with TARANTULA mandate — replaced by the Replit Agent and Workspaces.

- **Supabase-as-everything** (DB + auth + Edge Functions + storage) — replaced by Postgres + Drizzle (DB), Clerk (auth), Express (compute), Replit App Storage (object storage).
- **Sanity CMS** for marketing — landing and [/pricing](#) are rendered in-app; no third-party CMS dependency.
- **Vercel** as the hosting target — replaced by Replit deployments.
- **Direct Edge-Function-to-Anthropic** calls — replaced by the Replit AI Integrations proxy with a provider-switching layer.
- **Implicit response shapes** — replaced by an OpenAPI-first contract with generated Zod and React Query.

5.7 Risks the current PDD now owns that the prior PDD did not

Risk	Mitigation in the current build
Drift between certified MVP PDD and generated CODEBASE_BUNDLE	PFP engine + F8 hard drift gate (<code>`acknowledgeDrift`</code> override only)
Self-contradictory model output bypassing drift gate	Server-side recomputation of <code>`counts`</code> and <code>`verdict`</code> from <code>`findings`</code> before persistence
Busy session burying the relevant PFP report past a row window	<code>`latestPfpForMvp`</code> filters by <code>`artifact_content->>'sourceMvpPddArtifactId'`</code> with <code>`LIMIT 1`</code>
SSRF via user-supplied AISE / AISE_BUILD URLs	<code>`lib/badges.ts#headOk`</code> rejects RFC1918, loopback, metadata, link-local, CGNAT, multicast, and non- <code>`https`</code>
Ingestion credit double-spend on retry	Atomic claim with <code>`UPDATE ... FOR UPDATE SKIP LOCKED`</code> ; release on any error path
Cartridge wizard submitted without real scope	Hard server-side validation: "e 80 chars scope + "e 12 chars outcome + non-empty project name; <code>`code: "SCOPE_REQUIRED"`</code>
Cartridge context not being treated as authoritative by the LLM	Prepended to every system prompt, fenced as <code>`=== CARTRIDGE CONTEXT (authoritative · do not contradict) ===`</code> , scope first
Stripe webhook replays double-charging credits	<code>`stripe_webhook_events`</code> idempotency table; rollback-on-throw so legitimate retries still process
Account delete leaving orphan Stripe subs	External-first ordering: Stripe cancel !' Clerk delete !' local cascade
Cross-provider regressions invisible until production	Nightly cross-provider GitHub Action with byte-level fixture diff + auto-issue

5.8 What was deferred in the prior PDD and is now in scope

The prior PDD deferred 32 prompts as "CLASS C · upgrade-triggered." The current build folds most of the spirit of those deferrals into shipped features (cartridge context, ingestion, drift detection,

codebase scaffolding, per-engine telemetry, Context Craft badges) rather than holding them as a Sprint-2 backlog.

PART 6 — CERTIFICATION BLOCK

6.1 Verification surface

- Public verification URL: each SPARTAN-certified MVP PDD carries a URL that resolves to a verifier endpoint.
- AISE / AISE_BUILD badges: claimed via URL that is HEAD-checked through the SSRF-hardened verifier.
- Stripe webhook events: replayable by [event_id](#) for audit.
- Per-engine runs: every Claude call recorded in [harness_engine_runs](#) for post-hoc analysis.
- PFP reports: durable artifacts referencing both the MVP PDD and the CODEBASE_BUNDLE they were computed against.

6.2 Document status

This PDD is the current-state reference for the ATANDA Command Centre as of May 2026. It supersedes the prior MVP ATLAS PDD for any planning, investor, or engineering decision. The prior PDD remains valid as a historical artefact of the original four-week MVP scope on the pre-replatform stack.
